

Angular Moderno

07 - Estrutura do Projeto Angular Moderno



Angular Moderno

07 - Estrutura do Projeto: Arquitetura Base

Aprenda a organizar seu projeto Angular de forma profissional com uma arquitetura escalável baseada em **core**, **shared** e **features** — padrão usado em projetos reais de mercado.

Estrutura do Projeto

Nesta aula vamos aprender a organizar um projeto Angular de forma profissional, saindo de uma estrutura simples de componentes para uma arquitetura baseada em domínios. Esse é um passo fundamental para qualquer desenvolvedor Angular que queira trabalhar em projetos reais.

Arquitetura

Separação por responsabilidade
com core, shared e features

Projeto

Aplicando no e-commerce em
construção

Prática

Passo a passo detalhado e sem
pular etapas

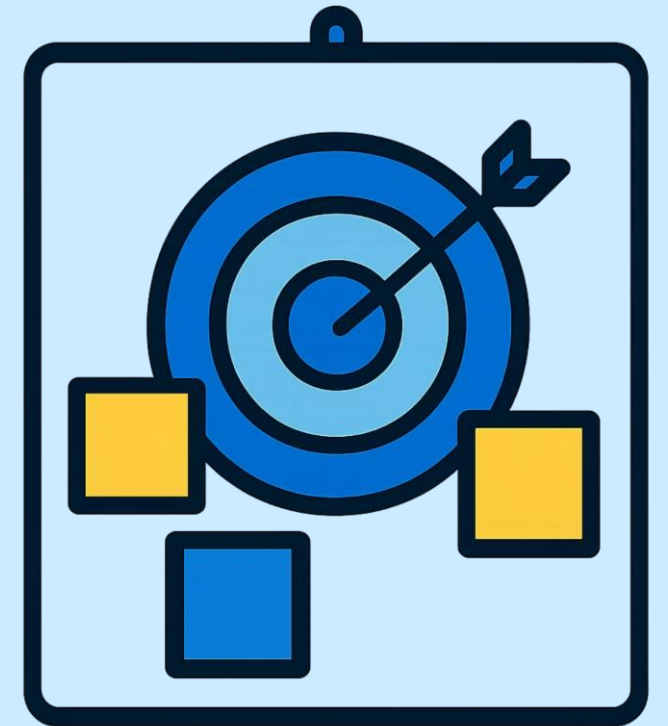
Objetivo da Aula

Ao final desta aula, você será capaz de organizar um projeto Angular de forma profissional, utilizando uma arquitetura moderna baseada em três pilares principais:

- 1 core/
Funcionalidades globais da aplicação
- 2 shared/
Elementos reutilizáveis entre features
- 3 features/
Funcionalidades do sistema por domínio

Vamos aplicar essa estrutura diretamente no projeto de e-commerce que estamos construindo ao longo do curso.

Nesta aula, vamos criar a estrutura, mas, inicialmente, utilizaremos apenas a pasta features.



Contexto: Onde Estamos

Nas aulas anteriores, evoluímos bastante! O projeto já possui uma estrutura funcional com os seguintes elementos implementados:

O que já temos

- Componente `produto` (filho)
- Componente `lista-produtos` (pai)
- Comunicação com `@Input` e `@Output`
- Renderização de lista com `@for`

O Problema

Toda a estrutura ainda está centralizada em uma única pasta `components/`. Isso funciona para projetos pequenos, mas **não é adequado para projetos reais** que crescem com o tempo.

```
src/app/  
  components/  
    produto/  
    lista-produtos/
```

O Problema da Estrutura Atual

A pasta `components/` funciona — mas não escala. Veja o que acontece quando o projeto cresce:

Dificuldade de Manutenção

Todos os componentes ficam misturados em um único lugar. Encontrar o arquivo certo vira um trabalho demorado conforme o projeto cresce.

Falta de Organização por Domínio

Sem separação por domínio, fica difícil entender o que pertence a qual funcionalidade do sistema.

Problemas em Times Grandes

Em projetos com múltiplos desenvolvedores, a ausência de uma arquitetura definida gera conflitos, duplicação de código e retrabalho.

⚠ Uma pasta `components/` genérica é um sinal de alerta em projetos Angular profissionais. Vamos resolver isso agora!

Arquitetura Moderna Angular

A solução é separar o projeto por **responsabilidade**. Cada pasta tem um papel claro e bem definido:



core/

Funcionalidades globais da aplicação: serviços singleton, configurações, interceptors, guards de autenticação. Carregado uma única vez.



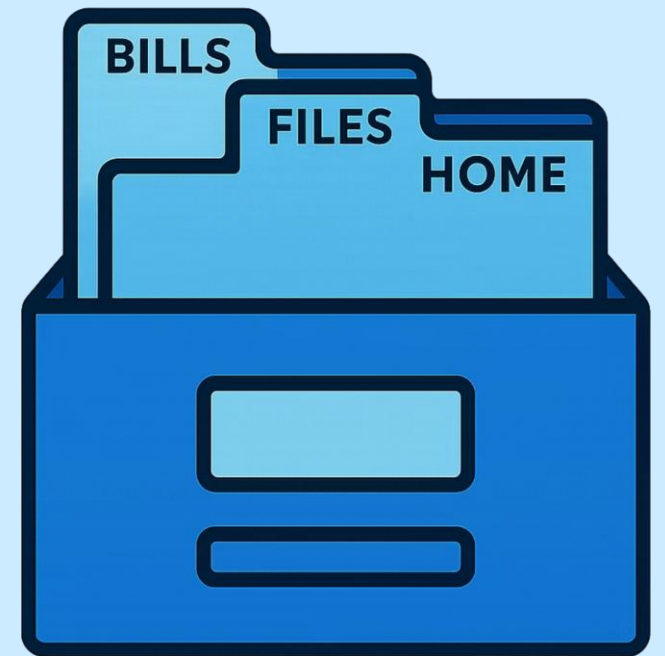
shared/

Componentes, pipes e utilitários reutilizáveis entre diferentes features. Tudo que pode ser compartilhado.



features/

Funcionalidades do sistema organizadas por domínio. Cada feature é independente e autocontida.



Entendendo Cada Pasta em Detalhes

core/

- Serviços globais (ex: AuthService)
- Interceptors HTTP
- Guards de rota
- Configurações de ambiente
- Carregado **uma única vez** na aplicação

shared/

- Componentes reutilizáveis (ex: botões, modais)
- Pipes customizados
- Diretivas utilitárias
- Interfaces e tipos compartilhados

features/

Cada feature representa uma funcionalidade do sistema:

```
features/  
  produtos/  
    produto/  
    lista-produtos/  
  carrinho/  
    item-carrinho/  
    resumo-carrinho/  
  checkout/  
    formulario/  
    confirmacao/
```

Cada feature é **independente** e agrupa tudo relacionado ao seu domínio.

Aplicando no Nosso E-Commerce

Agora que entendemos a arquitetura, vamos identificar onde cada componente do nosso projeto se encaixa:

Componente **produto**

Exibe as informações de um produto individual.

Pertence à feature **produtos**.

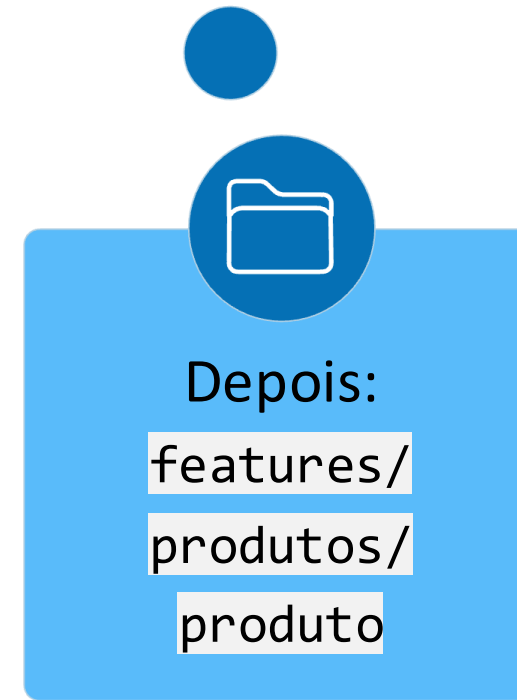
Componente **lista-produtos**

Renderiza a lista de produtos usando **@for**.

Pertence à feature **produtos**.

A regra é simples: se o componente faz parte do domínio de "produtos", ele vai para **features/produtos/**.

Conforme o e-commerce crescer, criaremos features como **carrinho/** e **checkout/**.



Parte Prática — Passo a Passo

Vamos realizar a migração da estrutura do projeto. São **9 passos** detalhados . Siga exatamente na ordem apresentada para evitar erros.



Passo 1 — Parar o Servidor

Antes de qualquer alteração na estrutura de pastas, é **obrigatório parar o servidor de desenvolvimento**. Mover arquivos com o servidor rodando pode causar erros de compilação e comportamentos inesperados.

Como parar o servidor

No terminal integrado do VSCode (onde o `ng serve` está rodando), pressione:

CTRL + C

O terminal ficará disponível novamente para novos comandos.

Como confirmar que parou

Após pressionar `CTRL + C`, o terminal deve mostrar o cursor piscando, indicando que está pronto para receber novos comandos.

✅ Servidor parado com sucesso! Agora é seguro mexer na estrutura de pastas.

Atenção

Em alguns sistemas, pode ser necessário pressionar `CTRL + C` duas vezes e confirmar com **S** ou **Y**.

Passo 2 — Criar Pastas Base

Dentro de `src/app/`, vamos criar as três pastas que formam a espinha dorsal da nossa arquitetura:

Como criar no VSCode

1. No painel **Explorer** (lateral esquerda), localize a pasta `src/app/`
2. Clique com o botão **direito** sobre a pasta `app/`
3. Selecione **"New Folder"** (Nova Pasta)
4. Digite o nome da pasta e pressione **Enter**
5. Repita o processo para as três pastas

Estrutura após este passo

```
src/  
  app/  
    core/      ← nova  
    shared/    ← nova  
    features/  ← nova  
    components/ ← ainda existe  
    app.ts  
    app.html
```

- ❏ Crie as três pastas: `core`, `shared` e `features`. Neste momento, deixe-as vazias. Vamos preenchê-las nos próximos passos.

Passo 3 — Criar a Feature "produtos"

Agora vamos criar a subpasta `produtos` dentro de `features/`. Ela representará o domínio de produtos do nosso e-commerce.

Como fazer

1. No Explorer do VSCode, localize a pasta `features/` que acabamos de criar
2. Clique com o botão **direito** sobre `features/`
3. Selecione **"New Folder"**
4. Digite `produtos` e pressione **Enter**

Resultado

```
src/  
  app/  
    features/  
      produtos/ ← criada agora  
  core/  
  shared/  
  components/  
    produto/  
    lista-produtos/
```

No futuro, criaremos outras features como `carrinho/` e `checkout/` dentro de `features/`. Por enquanto, focamos em `produtos/`.

Passo 4 — Mover os Componentes

Agora vamos mover os componentes existentes para dentro da feature `produtos`. No VSCode, você pode arrastar e soltar as pastas diretamente no Explorer.

O que mover

Mover a pasta `produto`:

De: `components/produto/`

Para: `features/produtos/produto/`

Mover a pasta `lista-produtos`:

De: `components/lista-produtos/`

Para: `features/produtos/lista-produtos/`

Estrutura após mover

```
features/  
  produtos/  
    lista-produtos/  
      lista-produtos.ts  
      lista-produtos.html  
      lista-produtos.css  
    produto/  
      produto.ts  
      produto.html  
      produto.css
```

✅ Dica: No VSCode, arraste a pasta `produto` e solte sobre `features/produtos/`. Confirme se o VSCode perguntar sobre mover o arquivo.

Passo 5 — Remover a Pasta "components"

Com os componentes movidos, a pasta `components/` deve estar vazia. Vamos removê-la para manter a arquitetura limpa.

1 Verificar se está vazia

No Explorer do VSCode, expanda a pasta `components/` e confirme que não há arquivos restantes. Ela deve estar completamente vazia.

2 Buscar referências antigas

Use `CTRL + SHIFT + F` para buscar por `components/` no projeto inteiro. Se aparecer algum resultado, anote para ajustar nos próximos passos.

3 Deletar a pasta

Clique com o botão direito sobre `components/` → **Delete**. Confirme a exclusão. A pasta antiga não faz mais parte da arquitetura.

⚠ Nunca deixe a pasta `components/` convivendo com a pasta `features/`. Isso gera confusão e fere o padrão arquitetural que estamos adotando.

Passo 6 — Ajustar o Import no app.ts

Agora que os componentes estão em um novo caminho, precisamos atualizar o arquivo `app.ts` para apontar para o local correto.

❌ ANTES (caminho antigo)

```
import { ListaProdutos }  
  from './components/lista-produtos/  
        lista-produtos';
```

⊗ Este caminho não existe mais! O Angular não vai encontrar o componente e a aplicação vai falhar na compilação.

✅ DEPOIS (caminho novo)

```
import { ListaProdutos }  
  from './features/produtos/  
        lista-produtos/  
        lista-produtos';
```

☑ Este é o novo caminho correto após a migração dos componentes para a pasta `features/produtos/`.

O Angular precisa conhecer o novo caminho do componente para importá-lo corretamente. Salve o arquivo após a alteração com `CTRL + S`.

Passo 7 — Validar o app.html

O arquivo `app.html` é o template principal da aplicação. Neste passo, vamos verificar se o template está correto — ele **não precisa de alteração**, pois a tag do componente não muda.

 Conteúdo esperado do app.html

```
<h1>Minha Loja</h1>
<app-lista-produtos></app-lista-produtos>
```

 Por que não muda?

O arquivo HTML usa o **seletor** do componente (`app-lista-produtos`), não o caminho do arquivo. O seletor não muda quando movemos o componente de pasta.

O que mudou foi apenas o **import no app.ts**, que agora aponta para o novo local do arquivo TypeScript.

 Regra importante: o HTML usa seletores, o TypeScript usa caminhos de arquivo. São conceitos distintos — seletores não mudam com a movimentação de arquivos.

Passo 8 — Mover a Pasta Pipes

Em atividades anteriores criamos a pasta `pipes`, que contém 2 arquivos: `preco-formatado-pipe.spec.ts` e `preco-formatado-pipe.ts`.

Desafio: Esta pasta deve ser movida para dentro de qual pasta recém criada? (`core`, `features` ou `shared`)

Você acertou se respondeu `shared`, pois esta pasta é o local ideal para elementos reutilizáveis, como pipes, que podem ser usados em qualquer parte da aplicação.

O que mover

Mover a pasta `pipes`:

De: `app/pipes`

Para: `shared/pipes/`

Estrutura após mover

```
app/  
  shared/  
    pipes/  
      preco-formatado-pipe.spec.ts  
      preco-formatado-pipe.ts
```

Passo 9 — Reiniciar o Servidor

Todos os ajustes foram feitos. Agora é hora de reiniciar o servidor e verificar se tudo está funcionando corretamente.

Comando para reiniciar

No terminal do VSCode, execute:

```
ng serve
```

Aguarde a mensagem de compilação bem-sucedida antes de abrir o navegador.

O que observar

- O terminal deve compilar **sem erros**
- Acesse `http://localhost:4200`
- A lista de produtos deve aparecer normalmente
- Nenhuma funcionalidade deve ter sido perdida

✔ Se o servidor compilar sem erros e a aplicação abrir normalmente, a migração foi um sucesso!

Resultado Esperado

Se todos os passos foram seguidos corretamente, o projeto deve estar exatamente igual ao que estava antes — mas agora com uma arquitetura profissional!



Lista Funcional

A lista de produtos continua sendo exibida corretamente no navegador, sem nenhuma perda de dados ou funcionalidade.

Comunicação Intacta

Os `@Input` e `@Output` entre `lista-produtos` e `produto` continuam funcionando perfeitamente.

Arquitetura Profissional

O projeto agora segue o padrão de mercado com separação por domínio, pronto para crescer.

✅ Nenhuma alteração foi feita na lógica — apenas na organização do projeto.

O Que Fizemos — Evolução da Estrutura

Veja a transformação completa que realizamos nesta aula:

Antes — Estrutura Simples

```
src/app/  
  components/  
    produto/  
      produto.ts  
      produto.html  
  lista-produtos/  
    lista-produtos.ts  
    lista-produtos.html  
  pipes/  
    preco-formatado-pipe.ts  
  app.ts  
  app.html
```

Funciona, mas não escala. Tudo misturado em um único lugar.

Depois — Estrutura Profissional

```
src/app/  
  core/  
  features/  
    produtos/  
      produto/  
        produto.ts  
      lista-produtos/  
        lista-produtos.ts  
  shared/  
    pipes/  
      preco-formatado-pipe.ts  
  app.ts  
  app.html
```

Separação por domínio. Base pronta para crescer.

Conexão com o Mercado Real

Essa arquitetura não é teórica! Ela é amplamente adotada em projetos Angular corporativos ao redor do mundo.

Escalabilidade

Projetos que crescem de 5 para 50 componentes continuam organizados. Cada nova feature tem seu lugar definido.

Times Grandes

Múltiplos desenvolvedores podem trabalhar em features distintas sem conflito. A separação por domínio facilita o trabalho em paralelo.

Manutenção Facilitada

Quando há um bug em "produtos", você sabe exatamente onde olhar: `features/produtos/`. Sem precisar vasculhar o projeto inteiro.



Resumo Desta Aula

Parabéns! Você completou uma das aulas mais importantes do curso. Veja tudo o que realizamos hoje:

1

Criamos a Arquitetura Base

Pastas `core/`, `shared/` e `features/` criadas em `src/app/`

2

Organizamos a Feature Produtos

Criamos `features/produtos/` para agrupar os componentes do domínio

3

Movemos os Componentes

`produto` e `lista-produtos` agora vivem dentro da feature correta

4

Ajustamos os Imports

Atualizamos o `app.ts` com o novo caminho dos componentes

5

Removemos a Estrutura Antiga

Pasta `components/` deletada — zero convivência de padrões

6

Sistema Funcionando

Validamos que tudo continua operacional após a migração

Regras de Ouro – Angular Moderno

Leve estas regras para todos os seus projetos Angular:

- Nunca use pasta `components/` genérica
Componentes reutilizáveis vão em `shared/`. Componentes de domínio vão em `features/`.
- Sempre siga a arquitetura `core/shared/features`
Esse padrão é reconhecido e valorizado em entrevistas e projetos reais no mercado.
- Não misture padrões antigos e novos
Escolha uma abordagem e seja consistente. Misturar cria confusão e dificulta a manutenção.
- Sempre valide após refatorar
Reinicie o servidor e confirme que tudo funciona após cada mudança estrutural no projeto.

✅ Neste projeto, utilizamos arquitetura baseada em features desde o início.



Dúvidas

?

